Taller de Lexing y Parsing

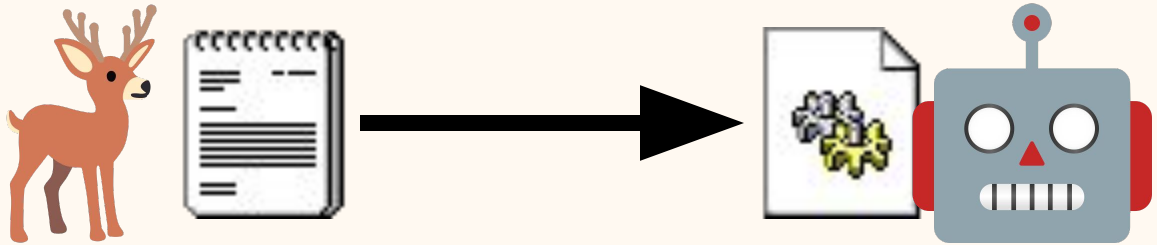
Segundo cuatrimestre, 2026



Motivación

Motivación

- Tenemos un **string** que representa información descrito en un formato particular que queremos pasar a una representación en memoria más fácilmente digerible.
- Dicho en crillo, tenemos **texto plano** y queremos pasarlo a un **objeto** que podamos usar en un lenguaje de programación.
- Como podemos lograr esto?
- Podríamos ir línea por línea e ir viendo sobre la marcha. No?



- **Sí**, y **no**. Hay formatos de texto donde esto es perfectamente aceptable, como el formato de configuración **INI**.
- En este caso, solo puede haber dos clases de líneas: líneas de **header**, y líneas con **clave** + **valor**.
- Es totalmente aceptable no escribir nada más complejo que un algoritmo que itere línea a línea, que parta la línea con =, y que agregue la clave y valor en una línea a un diccionario en memoria. Pero queremos hacer algo más complejo que esto, y además, queremos ser tolerantes de desviaciones en estilo de escritura.

[Taller]
Organización=”ComCom”
Año=”2026”
Título=”LexersYParsers”

Motivación

- Piensen en su lenguaje de programación favorito. Probablemente, alguna vez escribieron texto de una forma no-estándar, pero pensada para mejorar la lectura.
- O eso creemos, por lo menos. Y el lenguaje debe estar a la altura de poder tolerar estos casos no-estándar, en vez de forzar el código al ser escrito de una manera muy particular.

```
void foo()
{
  if (a)
  {
    printf(“hello, world\n”);
  }
}
```

```
void foo() {
  if (a) {
    printf(“hello, world\n”);
  }
}
```

Motivación

- Piensen en su lenguaje de programación favorito. Probablemente, alguna vez escribieron texto de una forma no-estándar, pero pensada para mejorar la lectura.
- O eso creemos, por lo menos. Y el lenguaje debe estar a la altura de poder tolerar estos casos no-estándar, en vez de forzar el código al ser escrito de una manera muy particular.

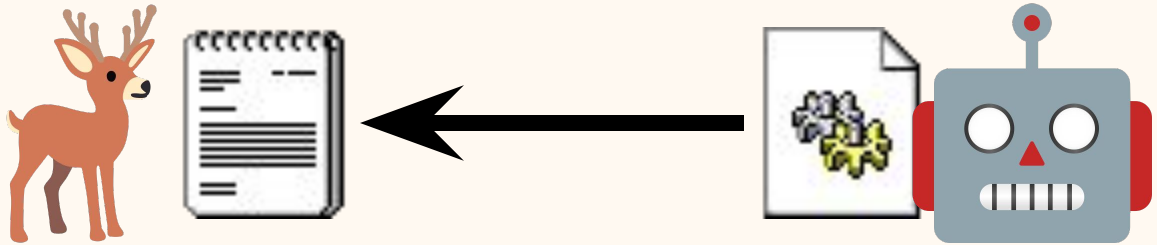
```
void foo() {
  int a = 0;

  bar(a); a++;
  bar(a); a++;
}
```

```
void foo() {
  int a = 0;

  bar(a);
  a++;
  bar(a);
  a++;
}
```

- Entonces tenemos que pensar un algoritmo para esto. Quizá podríamos abstraernos de la representación en texto plano y pasar a algo más concreto, como **símbolos** o **tokens**. ¿No?
- Los **símbolos/tokens** después nos permiten reconstruir lo que el string dice (o quiere decir), y es más fácilmente iterable para un algoritmo que chequear un string contra un string permanentemente.
- Eso es lo que vamos a estar haciendo hoy; un **lexer** que lee un string y devuelve una lista de símbolos, y un **parser** que toma una lista de símbolos y devuelve una lista de objetos.



Motivación

- Nuestro formato de texto que vamos a usar de ejemplo hoy es el **lenguaje de especificación** de **Algoritmos y Estructuras de Datos**. Volvió para atormentarlos de nuevo.
- Vamos a escribir un **lexer** + **parser** completo que abarque cualquier TAD, procedimiento, predicado y auxiliar que se imaginen.
- Antes, un mini-repaso del lenguaje.





```
/* TAD */
TAD Vector {
    obs x;
    obs y;

    /* Procedimiento */
    proc length(in v: Vector): Real {
        requiere { True }
        asegura { res = raizCuadrada(v.x * v.x + v.y * v.y) }
    }

    /* Predicado/Auxiliar (usa un tipo de retorno, como proc) */
    pred/aux esNulo(v: Vector) {
        v.x = 0 and v.y = 0
    }
}
```

Dinámica del taller

Dinámica del taller

- Tienen una carpeta **taller-lenguajes** disponible donde van a encontrar los siguientes archivos y carpeta:
  - **test/**
  - **lexer.py/lexer_test.py**
  - **parser.py/parser_test.py**
  - **common.py**
  - **diapositiva.pdf**
- Cada archivo de Python contiene un esqueleto de código con pseudocódigo y asistencia para saber qué hacer o que se espera que hagan.
- El archivo **common.py** incluye enums y clases predefinidas para que los asista en el taller. **EMPIECEN** leyendo este código para no redefinir nada.
- La dinámica del taller es que van a tener que implementar tanto el lexer como el parser asegurándose que su solución es correcta corriendo el test suite en cada archivo.
- Van a tener un buen tiempo para cada fase, pero no mas de 1 hora.

Dinámica del taller

- En la carpeta test/, van a poder encontrar un número de archivos ejemplo para testear su lexer/parser.
- Al correr **lexer_test.py** O **parser_test.py** con el comando *python3 lexer*_test*.py/parser*_test*.py*, se van a ejecutar los **test suite** del lexer/parser.
- Al correr **lexer_test.py** O **parser_test.py** con el comando *python3 lexer*_test*.py/parser*_test*.py {archivo}*, se ejecuta el **lexer/parser** con el archivo de texto pasado por argumento (ya leído como string) a la función principal del archivo (**lexer()/parser()**).

- Ejemplo:
  - **Test suite de lexer**: *python3 lexer*_test*.py*
  - **Archivo**: *python3 lexer_test.py aux_distancia.py*

  NO es necesario agregar el prefijo test/ a un archivo que quieren leer, el código ya lo hace por ustedes.
- Su **solución** tiene que estar en el archivo **lexer.py/parser.py**

Lexer

- Dijimos entonces que queremos escribir un **algoritmo** que permita tomar un **string** y devolver una lista de **símbolos**.
- Podríamos empezar con cualquier objeto de nuestro lenguaje, pero sería sabio empezar con el más simple; el **aux** (auxiliar).
- Conceptualmente, queremos iterar línea por línea, carácter por carácter, decidiendo qué hacer al encontrar un espacio o un carácter especial (**+**, **-**, **=**). ¿No?
- Básicamente, sí.

- Miremos entonces un auxiliar de ejemplo.

aux sumarUno(e: Entero): Entero = e + 1

- Observamos que sigue una estructura definida: aux, un nombre, lista de parámetros/argumentos, tipo de retorno, un igual, y el cuerpo del auxiliar.
- Entonces debería retornar una lista de símbolos del tipo:

['aux', Identifier("sumarUno"), '(', Identifier("e"), ':', Identifier("Entero"), ')', ':', Identifier("Entero"), '=', Identifier("e"), '+', Number(1)]

- Fua. Una bocha. Pero es fiel a la representación de texto. ¿No? Ejercicio para el lector.

- Un ejemplo de pseudocódigo sería:

```
                                obs foo: Real
                                obs bar: Real

por cada línea:
    por cada carácter:
        si el carácter es un carácter reservado:
            si la lista de caracteres no está vacía:
                convertir lo que esté en la lista temporal en un token
            agregar a la lista de tokens
        si el carácter es un espacio:
            convertir lo que esté en la lista temporal en un token
        si el carácter es cualquier otro:
            agregar a la lista temporal de caracteres

si la lista de caracteres no está vacía:
    convertir lo que esté en la lista temporal en un token
```

Parser

- Ahora que tenemos nuestra lista de **tokens**, tenemos que convertir uno o más **tokens** en algo un poco más cercano a lo que nosotros manejamos en programación: **clases**, u **estructuras de datos** que **representan** nuestras **construcciones del lenguaje**.
- Nosotros sabemos que, si iteramos sobre una lista de **tokens**, y el elemento actual dentro de la lista de **tokens** es un *X*, entonces deberíamos entrar en una **rutina/función** de código que asuma que el elemento siguiente tiene que ser un *Y*.
- Esto se llama un parser de **recursión descendente**, ya que empezamos al principio de todo y vamos iterando hacia abajo, recursando cuando sea necesario (es posible parsear una función definida dentro de una función, etc.)

```
obs x: Real
    ↔
function parse(t: TokenBuffer): Observer {
  token := t.peek();

  while (token != null) {
    if (token = Token::Observer) {
      parse_observer(t);
    } else if (...) {
      ...
    } else {
      error(“invalid token {token}”);
    }

    token := t.peek();
  }
}
```

```
obs x: Real
```

↔

```
function parse_observer(t: TokenBuffer): Observer {
  // obs
  t.want(TokenKind.Observer);
  // x
  name := t.want(TokenKind.Identifier);

  // :
  t.want(Token.Colon);
  // Real
  type := t.want(TokenKind.Identifier);

  return Observer {
    name := name,
    type := type
  };
}
```

- Sin embargo, hay un caso muy muy particular de algo que queremos parsear, pero que **no** tiene **estructura** bien **definida**.
- Es decir, nosotros humanos podemos detectar errores de sintaxis si las hay, pero siguen un orden de operaciones muy particular y pueden tomar cualquier forma y seguir siendo válido.
- ¿Se imaginan de qué estoy hablando?
- Piensen...

(1 + 1) / 2

↔

1

↔

2 - 1

↔

-1 + 2

- Sí, las **expresiones**. Son la más grande excepción dentro de un parser, ya que siguen reglas escritas en piedra...pero pueden tomar cualquier forma y seguir siendo válidas.
- Pero hay una herramienta que nos puede ayudar: un parser de Pratt (o **Pratt parser**).
- Es un parser pensado para parsear **expresiones** como las que conocemos de toda la vida: constantes, operaciones prefijas, infijas, postfijas, indexaciones, etc.
- Dato de color: la indexación ES una operación postfija, lo mismo con la invocación!

- Podemos clasificar las expresiones como:
  - **Atómicas**: son **expresiones atómicas** que denotan **valores**: son indivisibles.
    - Ejemplos: "hola", 1337.0, 67, [1, 2, 3]
  - **Operaciones**: denotan **operaciones** de **prefijo**, **infijo**, o **postfijo** que involucran otras **expresiones**, que pueden ser en sí también **operaciones**, o **atómicas**.
    - Ejemplo infijo: 1 + 1, 4 / 2, (2 + 2) / 4
    - Ejemplo prefijo: -1, -1337, not True
    - Ejemplo postfijo, a.x, array[1], foo()

- Nos encontramos con un problema con las operaciones, que es que nosotros tenemos un **orden de operaciones** mental (**PEMDAS**, o *Parenthesis*, *Exponent*, *Multiplication*, *Division*, *Addition*, *Subtraction*) para poder desambiguar en qué orden debemos evaluar una **expresión** en la matemática.
  - Ejemplo: 2 + 4 * 2 / 4
    - Es 4?
    - Es 3?
- Necesitamos, de alguna manera, poder modelar este orden implícito en programación.
- ¿Como podemos hacer?

- Cada operador (ya sea infijo, prefijo, postfijo) tiene asociado un "poder" de ligamiento, un valor numérico que representa que tan fuerte puede asociar operandos cercanos a sí mismo.

Operador	P. de ligamiento (izq./der.)
+	← 1, 2 →
-	← 1, 2 →
*	← 3, 4 →
/	← 3, 4 →

- Entonces, al evaluar de nuevo la siguiente **expresión**:

*2 + 4 * 2 / 4*

- Cada operador **liga** de tal manera:

*2 + (4 * 2) / 4*

*2 + ((4 * 2) / 4)*

2 + (8 / 4)

2 + 2 = 4

- Un ejemplo de pseudocódigo sería:

```
token ← [proximo token]
exp_l ← null

Si token es una expresión atómica:
  exp_l ← [Guardo la expresión atómica con el valor de token]
Si token es un operador:
  power ← [Busco el poder de ligamiento del token-operador prefijo]
  exp_r ← [Recurso en la función con power]
  exp_l ← [Guardo la expresión de tipo prefijo, con el op. token y exp_r]
Si no:
  error("{token} no es un valor/operador válido.")
```

- Un ejemplo de pseudocódigo sería:

```
while True:
    op ← [Miro el proximo token, sin sacarlo]
    Si op no es nulo:
        Si op no es un operador:
            error(“{op} no es un operador valido.”)
    Si no:
        break

    power ← [Busco el poder de ligamiento del token-operador postfijo]
    Si power no es nulo:
        Si power < power_parameter:
            break
        [...continua...]
```

- Un ejemplo de pseudocódigo sería:

```
Si power no es nulo:
     Si power < power_parameter:
          break

     [Finalmente poppeo el token del buffer]
     exp_l ← [Guardo la expresión de tipo postfijo, con el op, exp_l]
     continue

power ← [Busco el poder de ligamiento del token-operador infijo]
Si power no es nulo:
     Si power.L < power_parameter:
          break
     [Finalmente poppeo el token del buffer]
     exp_r ← [Recurso en la función con power.R]
     exp_l ← [Guardo la expresión de tipo infijo, con el op, exp_l, exp_r]
     continue
```

Cierre

Cierre

- Hoy aprendimos a pasar **texto plano**, a una secuencia de **símbolos/tokens**, para después pasarlos a un **objeto** cuantificable en **memoria**, el cual podemos **inspeccionar** como haríamos con cualquier otro dato.
- Dejamos como ejercicio para el lector implementar **parsing** de **expresiones** complejas en los aux/pred/proc en sus casas.
- Para muchos casos de uso, acá termina la historia. Sin embargo, si estuviéramos creando un **lenguaje de programación**, acá es donde observaríamos que **funciones** y **estructuras de datos** tenemos... para pasarlos a **código compilado**.
- El lenguaje de programación **Lua** es muy amigable para la lectura, y es un lenguaje muy pequeño también. Pueden investigar como pasan el **código parseado** a **código ejecutable** (en una **máquina virtual**, por supuesto).

Referencias/lectura adicional:

- Lexer de Lua
- Parser de Lua
- Lexer/Parser de TinyCCompiler
- Interprete de Brainfuck
- Pratt Parser

Fin